Reviewer 1

Comments

Reviewer 1 - Initial submission, 25/09/2026
The manuscript presents HiveWatch, a framework-agnostic observability toolkit for federated learning (FL). The software is practically motivated, and the integration with multiple FL frameworks and monitoring backends is interesting. The manuscript is generally well organized. I have a few suggestions that may help further improve the quality and clarity of the paper.
1) The motivation section could be further strengthened by providing a clearer comparison between HiveWatch and existing FL monitoring or experiment-tracking tools. In particular, a brief table comparing framework independence, FL-native metrics, staleness monitoring, failure tracking, communication statistics, and multi-backend support would help readers better understand the main advantages of HiveWatch.
2) The manuscript highlights heterogeneous resources, communication costs, failures, and staleness as important challenges in practical FL systems. The authors may wish to further enrich this discussion by considering recent studies on asynchronous and unreliable FL systems, for example: “Adaptive Semi-Asynchronous Federated Learning in Wireless Edge Networks,” IEEE TCOM 2024; and “Robust Federated Learning for Unreliable and Resource-limited Wireless Networks,” IEEE TWC, 2024. These studies may provide useful context for motivating the monitoring of staleness, resource heterogeneity, and client failures.
3) The manuscript defines “gradient divergence” as the standard deviation of per-client gradient norms. It would be helpful if the authors could briefly justify this definition, since variations in gradient norms may not fully capture differences in gradient directions. Alternatively, a more specific term such as “gradient-norm dispersion” may improve clarity.
4) Since HiveWatch records client-selection and participation-related information, the authors may also wish to discuss whether metrics such as participation frequency or client contribution history could be useful additions. This would be relevant for studying client-selection behavior in FL. A related reference that may be useful is “Exploring Representativity in Device Scheduling for Wireless Federated Learning,” IEEE TWC, 2024.
5) The GeoEmitter currently performs synchronous persistence, and the manuscript also notes that the snapshot cost may increase with scale. It would be helpful to briefly discuss the scalability of this design with a larger number of clients, as well as how HiveWatch handles a slow or temporarily unavailable emitter.

Reviewer 3

Comments

Reviewer 3 - Initial submission, 01/10/2026
The work introduces HiveWatch, an open-source, framework-agnostic observability toolkit fir unified monitoring and experiment tracking in federated learning (FL).

Strenghts:
1. HiveWatch presents a solution to a real-world challenge in FL. Framework-agnostic middleware with an FL-native schema successfully bridges execution frameworks (Flower, NVIDIA FLARE) with standard tracking backends (MLflow, Weights & Biases). in short, it tackles a higly relevant and interesting problems.

2. The experimental design is effective in presenting the acutal computational cost of the framework. By benchmarking internal bookkeeping against the synchronous GeoEmitter (JSONL and map snapshot writes), the authors show their performance claims. A mean overhead of ~3 ms per call (and ~64 ms for a 20-client round) shows the method's effectiveness.

3. The results are robust, well-documented, and reproducible. It is good to see the 5-runs with different seeds showing the robustness of the experimetns.

Weaknesses:

1. The manuscript slightly overstates the lack of native integration in existing tools. For example, NVIDIA FLARE already natively supports streaming metrics directly to MLflow, Weights & Biases, and TensorBoard without requiring custom client-side code changes. While HiveWatch's value lies in standardizing this across multiple frameworks, the claim that practitioners must currently manually build these connections from scratch is not entirely accurate for mature platforms.

2. The evaluation only benchmarks the server-side micro-overhead. It does not compare HiveWatch's latency against the native logging baselines of Flower, APPFL, or FLARE. Furthermore, the authors note that internal state is protected by a single threading.Lock. While acceptable for 20 clients, the lack of a high-concurrency baseline (e.g., 1,000+ simultaneous clients) leaves questions about potential lock contention in massive cross-device FL scenarios. It might be interesting to see the effectivness of the framework at scale

3. Excluding MLflow and W&B from the benchmarks is okay, but it means we do not get an empirical estimate of the end-to-end latency when using the primary third-party tools the paper argues.

4. Overlooks some of the recent related work and also a bit of a broader discussion on federated learning and its background. It will be interesting to add, specially for people who are not familiar to FL or are new to FL. Consider the following references.
[1] Que, Citong, and Faisal Nadeem Khan. "A scalable federated learning-based approach for accurate traffic prediction in edge computing-enabled metro optical networks." Computers & Industrial Engineering 203 (2025): 111004. [2] Zaland, Obaidullah, Sajib Mistry, and Monowar Bhuyan. "Guarding the middle: Protecting intermediate representations in federated split learning." 2025 IEEE International Conference on Big Data (BigData). IEEE, 2025. [3] Beltrán, Enrique Tomás Martínez, et al. "Fedstellar: A platform for decentralized federated learning." Expert Systems with Applications 242 (2024): 122861. [4] Zaland, Obaidullah, et al. "Federated learning for large-scale cloud robotic manipulation: Opportunities and challenges." 2025 International Conference on Machine Learning and Cybernetics (ICMLC). IEEE, 2025.